PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

CampusPath: Smart Navigation for Every Corner of Campus

Student / Team Information

Team Name: *Team #*	*Campus Navigators*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Verma, Preet –* *2510012046* *2510012046@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bhardwaj, Ishan* *2510370318* *2510370318@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Large university campuses can be difficult to navigate, especially for new students, visitors, and anyone searching for unfamiliar classrooms, laboratories, offices, or facilities. A normal map may show locations but does not explain the best route between two campus points. This project addresses that problem by representing the campus as a graph, where locations are nodes and connecting paths are edges. It applies data structures and graph algorithms in C++ to provide searchable locations, shortest-path navigation, nearby-location discovery, and useful route analysis through a simple campus map interface.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Students currently depend mainly on Google Maps, static campus maps, signboards, or asking other people for directions. These methods can work for major buildings, but they may be less convenient for smaller campus locations and do not demonstrate how navigation decisions are produced using core data structures and algorithms. The proposed system follows the graph-based approach used in the RudraxDave CampusNavigation project. Locations are represented as nodes, while connections form graph edges. Search and navigation functions can then use techniques such as autocomplete, edit distance, Dijkstra and Bellman-Ford shortest paths, DFS-based cycle detection, topological sorting, and TSP-based route optimization. This makes the project both a practical campus utility and a demonstrable DSA application.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The main goal is to develop a C++ campus-navigation application that models campus locations as a graph and demonstrates important data structures and algorithms through practical features. Initial milestones are: create and load the location dataset; represent nodes, coordinates, and neighbouring connections; implement location search and autocomplete; add edit-distance based name matching; implement shortest-path calculation using Dijkstra and compare it with Bellman-Ford; implement DFS-based cycle detection and topological sorting; add nearby-location search; and implement Travelling Salesperson Problem solutions using brute force, backtracking, and 2-opt. The final milestone is to connect these functions to a clear map-oriented interface, test them with sample cases, analyze time complexity, and prepare the final report and demonstration.

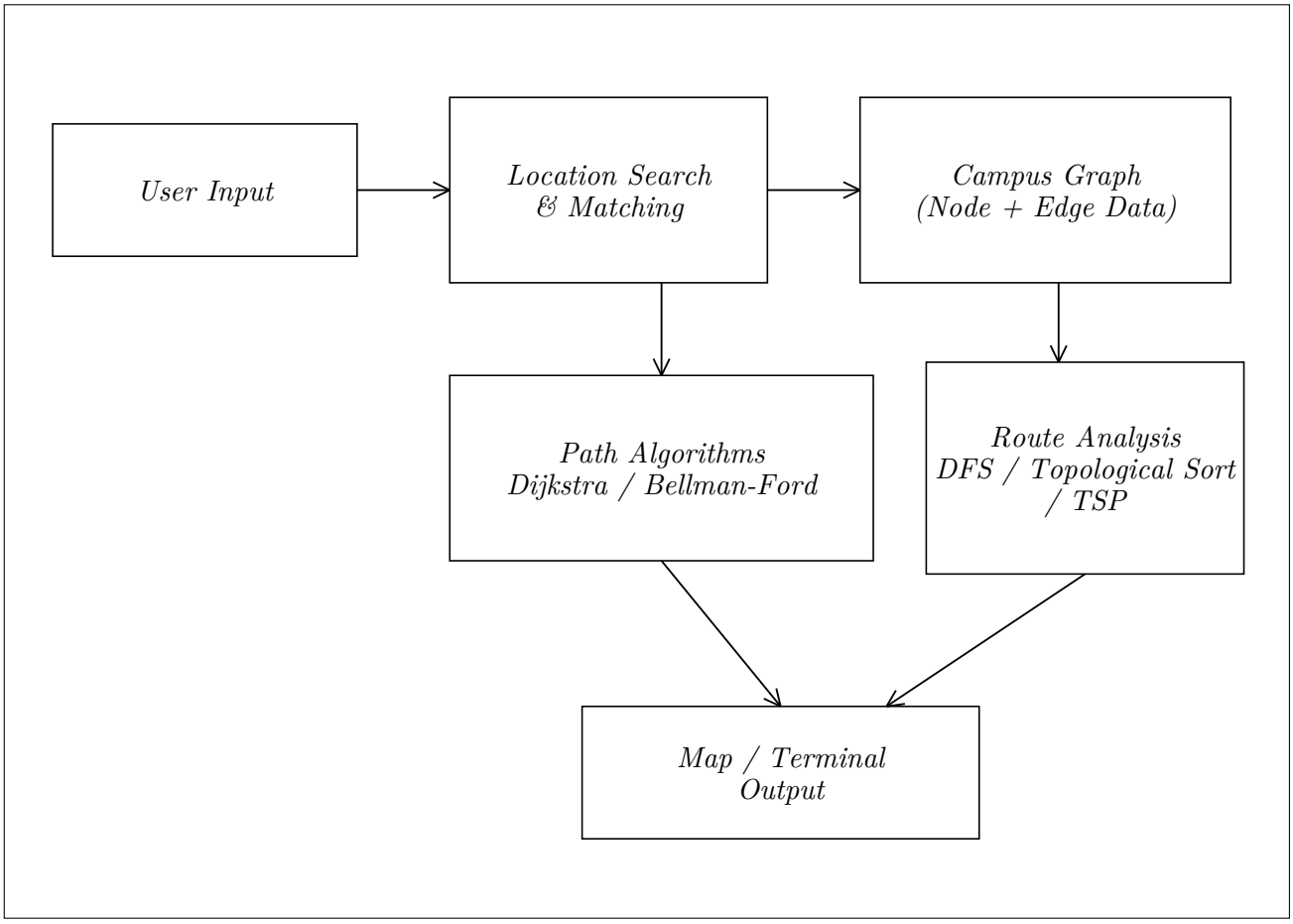
Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

The project will be implemented in C++ using standard STL containers such as vector, unordered_map, queue, stack, set, and related structures. The campus will be represented as a graph in which each location stores an identifier, name, coordinates, and connections to neighbouring locations. Input data will be loaded from structured files, while graph construction and helper functions will organize the dataset. Search will use string processing, autocomplete, and edit distance. Dijkstra and Bellman-Ford will calculate shortest routes, while DFS will support cycle detection and topological sorting. TSP will be explored through brute force, backtracking, and 2-opt. A lightweight terminal/map interface can present locations and routes. CMake and unit tests will be used to build and validate the application.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

The completed project will deliver a functional C++ campus-navigation prototype with a graph-based campus model, location search, autocomplete, shortest-path navigation, nearby-location discovery, and route-analysis features. It will demonstrate practical use of DSA concepts including graphs, hashing, vectors, queues, DFS, shortest-path algorithms, topological sorting, and TSP optimization. The deliverables will include the source code, input dataset, test cases, system architecture diagram, complexity analysis, screenshots or interface output, and a final project report. The result will provide a clear demonstration of how classical algorithms can solve a realistic campus-navigation problem.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

The campus dataset is assumed to contain valid location names, coordinates, and connections. Edge weights represent approximate travel distances. The prototype uses a fixed campus dataset and does not require live GPS, traffic data, or internet connectivity. Users are assumed to enter recognizable location names or close variations.

References

1. RudraxDave, "CampusNavigation" — GitHub: github.com/RudraxDave/CampusNavigation
2. cppreference, "C++ Standard Library": en.cppreference.com/w/cpp
3. T. H. Cormen et al., Introduction to Algorithms, MIT Press, 4th ed.